

POWERED BY IBM BOB

ReForge

Autonomous Repository Migration Workbench

From legacy Flask to FastAPI — with governed contracts, test parity, and audit evidence.

4

endpoints migrated

0

paths changed

0

HTTP methods changed

10/10

tests passing

Source: Flask + Flask-SQLAlchemy + SQLite → Target: FastAPI + SQLAlchemy 2.x + Pydantic v2 + SQLite

Why Manual Flask → FastAPI Migration Is Risky

A framework migration is not simply a syntax conversion.

P Engineers must preserve

- API routes and HTTP methods
- Request and response semantics
- Database tables, columns, types, and constraints
- Existing business logic
- Test behavior
- Cross-file dependencies

Broken baseline tests

- `conftest.py` imported `db` from the wrong location
- 2 of 5 legacy tests were initially non-runnable

Deprecated ORM patterns

- `Model.query.all()`
- `Model.query.filter_by(...).first()`

Framework coupling

- Flask application context & Blueprints
- `flask.request.get_json()` / `flask.jsonify()`

Additional migration risks

- Unpinned dependencies; no rollback on commits
- `/users` bypassed service layer; `config.py` was dead code

The core risk: *A migration can “work” while silently changing the API contract. ReForge treats compatibility as a contract, not an assumption.*

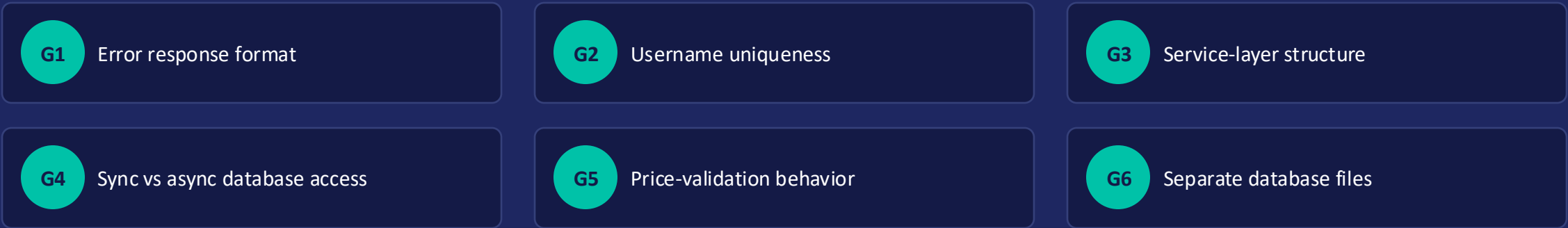
AI-Driven Build-Time Automation

ReForge turns repository migration into a controlled engineering workflow. IBM Bob does more than generate code:



HUMAN-IN-THE-LOOP WHERE AMBIGUITY MATTERS

Six architectural decisions were explicitly identified for human review:



THE 5-STAGE MIGRATION PIPELINE

Understand → Contract → Refactor → Test → Prove

01

UNDERSTAND

`recon.md`

Repository reconnaissance — maps 12 Python files, 4 API endpoints, 2 ORM models, 2 service modules, dependency/import graph, tests & assumptions, deprecated patterns, and migration risks before touching application code.

02

CONTRACT

`migration_contract
.md`

Defines what cannot change — endpoint invariants, response invariants, schema invariants, phase boundaries, allowed files, forbidden changes, and acceptance criteria.

03

REFACTOR

Controlled transformation across implementation phases: data layer, routes + services, FastAPI entry point, then tests + cleanup.

04

TEST

Preserves behavior — legacy assertions are ported using FastAPI TestClient, dependency overrides, and in-memory SQLite.

05

PROVE

`validation_report.
md`

Generates audit-grade evidence: full test suite, application startup, OpenAPI routes, endpoint smoke tests, schema parity, and a check for Flask-import / legacy .query leakage.

Same Behavior. Modernized Architecture.

BEFORE — legacy-flask-app/

```
legacy-flask-app/
├── app.py
├── config.py                # dead code
├── db/database.py
├── models/
│   ├── user.py
│   └── product.py
├── routes/
│   ├── auth.py
│   ├── users.py
│   └── products.py
├── services/
│   ├── auth_service.py
│   └── product_service.py
└── tests/ (conftest, test_*)
```

AFTER — fastapi-app/

```
fastapi-app/
├── main.py
├── db/database.py
├── models/
│   ├── user.py
│   └── product.py
├── schemas/
│   ├── user.py
│   └── product.py
├── routers/
│   ├── auth.py
│   ├── users.py
│   └── products.py
├── services/
│   ├── auth_service.py
│   └── product_service.py
└── tests/ (conftest, test_*)
```

Key architectural replacements

- `Flask.Blueprint` → `FastAPI APIRouter`
- `db.init_app()` → `SQLAlchemy engine + SessionLocal + get_db()`
- `Model.query` → `SQLAlchemy 2.x select()`
- `flask.request.get_json()` → `Pydantic request models`
- `flask.jsonify()` → `FastAPI response models`

ReForge audit layer — reforge/

recon.md	<i>what exists</i>
migration_contract.md	<i>what may change</i>
migration_log.md	<i>what changed & when</i>
human_review.md	<i>decisions needing human input</i>
validation_report.md	<i>what was proven</i>

Migration Freedom Stops Where Compatibility Begins

The migration contract is the source of truth for the transformation.

Endpoint	Method	Requirement
/auth/register	POST	Unchanged
/users	GET	Unchanged
/products	GET	Unchanged
/products	POST	Unchanged

STRICT RULES

No new paths

No path aliases

No version prefixes

No additional HTTP methods

Response contract example

```
POST /auth/register → 201
{ "message": "User created successfully",
  "user": { "id": int, "username": str, "email": str } }
```

Database invariants

- Tables remain users & products
- Column names & types unchanged
- Constraints unchanged
- No new columns / tables

Forbidden changes

- No feature additions
- No unrelated refactors
- No secret/config changes
- No frontend work

Also unchanged

- GET /users → 200 [{id, username, email}]
- GET /products → 200 [{id, name, price}]
- POST /products → 201 {id, name, price}

Every Step Is Traceable

ReForge does not treat migration as one large AI-generated change — work is divided into controlled phases with explicit boundaries.

A

PHASE A — Data Layer

SQLAlchemy 2.x database layer, User/Product models, Pydantic schemas, pinned dependencies. Also fixed the legacy confest.py import so the baseline became trustworthy.

B

PHASE B — Routes & Services

auth_service.py, product_service.py, auth.py, users.py, products.py — all four endpoints preserved.

C

PHASE C — Application Entry Point

main.py, router mounting, application lifespan, database initialization.

D

PHASE D — Tests, Cleanup & Audit

FastAPI test infrastructure, migrated test files, full test execution, final validation report.

AUDIT TRAIL

recon.md

what exists

migration_contract.md

what may change

migration_log.md

what changed & when

human_review.md

decisions requiring human input

validation_report.md

what was proven

The migration is reproducible and inspectable rather than a black box.

10/10 Tests Green. 4/4 Routes Preserved.

Test parity

Suite	Baseline	Final
Legacy Flask	3/5 runnable	5/5 passing
FastAPI	—	5/5 passing
Combined	3/5 runnable	10/10 passing

The two initially broken legacy tests were fixed before FastAPI implementation by correcting the `conftest.py` import.

Route parity

- ✓

/auth/register
- ✓

/users (GET)
- ✓

/products (GET)
- ✓

/products (POST)

4/4

endpoints migrated

0

paths changed

0

methods changed

Schema parity

users

- `id` — `INTEGER PK`
- `username` — `VARCHAR(80), UNIQUE, NOT NULL`
- `email` — `VARCHAR(120), UNIQUE, NOT NULL`

products

- `id` — `INTEGER PK`
- `name` — `VARCHAR(120), NOT NULL`
- `price` — `FLOAT, NOT NULL`

Additional validation

- ✓ Application imports cleanly
- ✓ OpenAPI schema resolves all 4 paths
- ✓ All 11 modules import cleanly
- ✓ No Flask imports remain
- ✓ No legacy `.query` usage remains
- ✓ Endpoint smoke tests: 10/10 PASS

Live Cloud Monitoring & Audit View

Instead of forcing a reviewer to inspect every repository file manually, the dashboard provides a visual entry point into the generated migration evidence.

Evidence sources

- Repository reconnaissance
- Migration contract
- Baseline information
- Migration execution records
- Human-review decisions
- Final validation report

KEY EVIDENCE SURFACED

4	endpoints migrated
0 / 0	paths & methods changed
10/10	tests passing
✓	schema parity verified

← → ↺

reforge-workspace-6f22rpobxnnfjzqimvgwwe.streamlit.app

🔍 ⭐ ⬇️ 📄 M ⋮

🔍

What's new

🌐 Climate Internship

📄 Elementary Linear A...

🔗 Open Source | MLH...

🌐 Outreachy Initial Ap...

👤 Your experience | Hi...

All Bookmarks

Share ☆ 🖨️ ⋮

ReForge - Repository Migration Workbench

Flask → FastAPI | Build-time automation with IBM Bob 2.0

Migration Contract

validation_report.md —
Flask → FastAPI Migration
Final Validation

migration_contract.md —
Flask → FastAPI Migration
Contract

Validation Report

Repository: `legacy-flask-app/` → `fastapi-app/`
Source of truth for recon: `reforge/recon.md`

Date: Phase D complete
Source: `legacy-flask-app/`

Manage app

LIVE DEPLOYMENT

<https://reforge-workspace-6f22rpobxnnfjzqimvgwwe.streamlit.app/>

ReForge: Migration With Proof

What we demonstrated

- 1 **Understand** the legacy repository before changing it
- 2 **Contract** the behavior that must remain unchanged
- 3 **Refactor** through controlled phase boundaries
- 4 **Test** against preserved legacy behavior
- 5 **Prove** the final state with machine-generated evidence

Modernizing legacy software should not require choosing between speed and control.

Final results

4/4

endpoints migrated

0

API paths changed

0

HTTP methods changed

10/10

tests passing

✓

schema parity verified

✓

full audit generated

SUBMISSION

GitHub: github.com/marihajaved33/reforge-workspace

Live Dashboard: reforge-workspace-6f22rpobxnnfjzqimvgwwe.streamlit.app

Powered by IBM Bob · ReForge — From legacy code to verified modernization.